

Linux Debugging — Hands-On Lab Manual



Prerequisites

- A Linux machine (Ubuntu 20.04+ / Debian 11+) — native, VM, or Raspberry Pi
- Root/sudo access
- GCC, Make, GDB, Valgrind installed
- Kernel headers installed (for kernel debugging labs)
- Basic C programming and Linux command-line familiarity

```
# Quick setup – run this first!
sudo apt update
sudo apt install -y build-essential gcc g++ make gdb valgrind strace ltrace \
    binutils elfutils linux-headers-$(uname -r) \
    linux-tools-common linux-tools-$(uname -r) \
    trace-cmd kernelshark \
    bpfcc-tools bpftrace libbpf-dev \
    wireshark tshark tcpdump nmap netcat-openbsd \
    crash kdump-tools kexec-tools \
    rsyslog systemd-journal-remote \
    python3 python3-pip git curl wget

# Install bcc (eBPF front-end)
sudo apt install -y bpfcc-tools python3-bpfcc 2>/dev/null || true

# Enable kernel tracing
sudo mount -t tracefs nodev /sys/kernel/tracing 2>/dev/null || true
```

🍷 **Raspberry Pi Users:** Most application debugging (GDB, sanitizers, Valgrind) and tracing (ftrace, perf) labs work on Raspberry Pi. Install tools with: `sudo apt install gdb valgrind strace ltrace trace-cmd linux-perf`. KGDB/KDB labs require serial console. eBPF support depends on kernel version (5.10+). Wireshark/tcpdump work fully.

⚠️ **Important:** Some kernel debugging techniques (KGDB, crash dumps, kexec) can halt or crash the system. Always test in a VM or on a dedicated test machine.

PART 1 — APPLICATION DEBUGGING

LAB 1: Application Debugging with GDB



Mission: "The Bug Hunter's Arsenal"

Story: A critical production application is crashing intermittently. The development team has been staring at the code for days. You've been called in as the debugging specialist. Your first tool: the GNU Debugger. Master it, and no bug can hide.



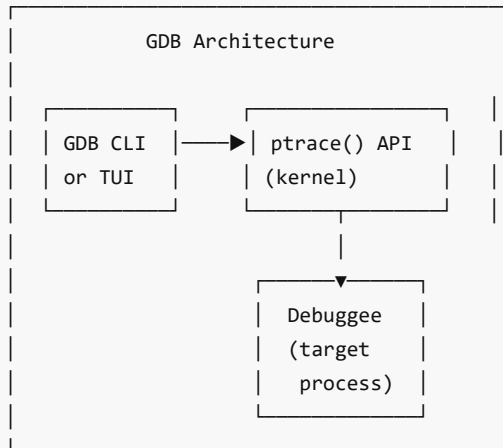
Background

GDB (GNU Debugger) is the most powerful debugging tool on Linux. It lets you:

- Set **breakpoints** to pause execution at specific lines or functions
- **Step** through code line by line or instruction by instruction
- **Inspect** variables, memory, registers, and the call stack

- **Modify** variables and memory at runtime
- Attach to **running processes**
- Analyze **core dumps** from crashed programs
- Debug **multi-threaded** applications

Key concepts:



Compile for debugging: **always use** `-g` (adds DWARF debug info):

```

gcc -g -O0 -o program program.c    # Debug build: symbols + no optimization
gcc -g -Og -o program program.c    # Debug-friendly optimization
gcc -ggdb3 -o program program.c    # Maximum GDB-specific debug info
  
```

❏ Exercise 1.1: GDB Basics — Breakpoints, Stepping, Inspection

Objective: Learn fundamental GDB commands with a multi-function program.

Step 1: Navigate to the lab directory:

```

cd code/debugging/lab01_gdb/
cat buggy_app.c
  
```

Step 2: Compile with debug symbols:

```
make
```

Step 3: Start GDB:

```
gdb ./buggy_app
```

Step 4: Learn essential commands — practice each one:

```

# Set breakpoints
(gdb) break main           # Break at main()
(gdb) break buggy_app.c:25 # Break at line 25
(gdb) break process_data   # Break at function name
(gdb) info breakpoints     # List all breakpoints

# Run the program
(gdb) run                  # Start execution
  
```

```

(gdb) run arg1 arg2          # Start with arguments

# When stopped at a breakpoint:
(gdb) next                   # Step OVER function calls (n)
(gdb) step                   # Step INTO function calls (s)
(gdb) finish                  # Run until current function returns
(gdb) continue                # Continue to next breakpoint (c)

# Inspect state
(gdb) print variable_name    # Print variable value
(gdb) print/x variable       # Print in hex
(gdb) print *ptr              # Dereference pointer
(gdb) print array[0]@10      # Print 10 elements of array

(gdb) info locals             # All local variables
(gdb) info args               # Function arguments
(gdb) backtrace               # Call stack (bt)
(gdb) frame 2                 # Switch to frame 2

(gdb) list                    # Show source code around current line
(gdb) list 1,50               # Show lines 1-50

# Memory examination
(gdb) x/10x $rsp              # 10 hex words at stack pointer
(gdb) x/s string_var          # Print as string
(gdb) x/20i $rip              # 20 instructions at instruction pointer

# Modify at runtime
(gdb) set variable x = 42     # Change a variable
(gdb) set *(int*)0x7fff1234 = 0 # Write to memory address

# Quit
(gdb) quit

```

Step 5: Practice a full debugging session — find the bugs in `buggy_app.c` :

```

(gdb) break main
(gdb) run
(gdb) next                   # Step through main
(gdb) print result           # Inspect values at each step
(gdb) step                   # Step into suspicious functions
(gdb) backtrace               # Where are we in the call chain?

```

Step 6: Answer in `answers.txt` :

1. What is the difference between `next` and `step` ?
2. How do you examine the call stack?
3. What does the `frame` command do?
4. How do you change a variable's value during debugging?

✓ **Checkpoint:** You should be comfortable setting breakpoints, stepping through code, and inspecting state.

🔗 Exercise 1.2: Conditional Breakpoints and Watchpoints

Objective: Use advanced breakpoint features to catch bugs that only trigger under specific conditions.

Step 1: Set conditional breakpoints:

```
(gdb) break process_data if count > 100
(gdb) break buggy_app.c:45 if ptr == NULL
(gdb) break compute if i == 999
```

Step 2: Use watchpoints — break when a variable changes:

```
# Hardware watchpoint: stops when value changes
(gdb) watch my_variable
(gdb) watch *0x7fff5678      # Watch a memory address
(gdb) rwatch my_variable    # Break on READ
(gdb) awatch my_variable    # Break on READ or WRITE
(gdb) info watchpoints
```

Step 3: Use catchpoints:

```
(gdb) catch throw            # Break on C++ exception throw
(gdb) catch syscall write    # Break on write() syscall
(gdb) catch signal SIGSEGV   # Break on segfault
```

Step 4: Automate with commands:

```
(gdb) break process_data
(gdb) commands
> print count
> print buffer[0]@5
> continue
> end
```

This prints `count` and first 5 buffer elements every time `process_data` is called, then continues automatically.

✅ **Checkpoint:** You can set breakpoints that trigger only under specific conditions.

🔗 Exercise 1.3: Debugging Segfaults with Core Dumps

Objective: Analyze crashed programs using core dump files.

Step 1: Enable core dumps:

```
# Enable core dumps (unlimited size)
ulimit -c unlimited

# Verify
ulimit -c

# Set core dump pattern (optional)
echo "core.%e.%p" | sudo tee /proc/sys/kernel/core_pattern
```

Step 2: Run the crashing program:

```
cd code/debugging/lab01_gdb/
./segfault_demo          # This will crash!
```

```
ls -la core*           # Core dump should appear
```

Step 3: Analyze the core dump with GDB:

```
gdb ./segfault_demo core.segfault_demo.*

# GDB opens with crash context:
(gdb) backtrace           # See where it crashed
(gdb) frame 0             # Go to crash frame
(gdb) list                # Show source at crash point
(gdb) info registers      # CPU register state at crash
(gdb) print *ptr          # Examine the bad pointer
(gdb) x/10x $rsp          # Stack contents at crash
```

Step 4: Practice the segfault analysis workflow:

1. backtrace → WHERE did it crash?
2. frame N → Go to the relevant frame
3. list → WHAT code was executing?
4. info locals → WHAT were the variable values?
5. print expr → Deep-dive into suspicious values
6. info registers → Low-level CPU state

Step 5: Answer in `answers.txt` :

1. What caused the segfault in `segfault_demo` ?
2. What was the value of the pointer that caused the crash?
3. What function and line number did the crash occur at?

✅ **Checkpoint:** You can analyze core dumps to determine crash root causes.

🔗 Exercise 1.4: Attaching GDB to Running Processes

Objective: Debug a process that's already running (live debugging).

Step 1: Start the long-running program:

```
cd code/debugging/lab01_gdb/
./long_running &
echo "PID: $!"
```

Step 2: Attach GDB:

```
sudo gdb -p $(pgrep long_running)
```

Step 3: Investigate the running state:

```
(gdb) backtrace           # Where is it right now?
(gdb) info threads        # List all threads
(gdb) thread 2            # Switch to thread 2
(gdb) bt                  # Backtrace for this thread

(gdb) break suspicious_func # Set breakpoint
(gdb) continue             # Let it run until breakpoint
```

```
(gdb) detach                # Disconnect without killing
```

Step 4: Use GDB in TUI (Text User Interface) mode:

```
gdb -tui ./buggy_app

# Or toggle TUI inside GDB:
(gdb) tui enable
(gdb) layout src           # Source code view
(gdb) layout asm           # Assembly view
(gdb) layout split         # Source + assembly
(gdb) layout regs          # Show registers
```

✅ **Checkpoint:** You can attach to and debug running processes.

🔗 Exercise 1.5: Multi-threaded Debugging with GDB

Objective: Debug race conditions and thread-specific bugs.

Step 1: Examine the multi-threaded program:

```
cd code/debugging/lab01_gdb/
cat thread_debug.c
make
```

Step 2: Debug with thread awareness:

```
gdb ./thread_debug
(gdb) break worker_thread
(gdb) run

# When breakpoint hits:
(gdb) info threads          # List all threads with states
(gdb) thread 3              # Switch to thread 3
(gdb) bt                    # Backtrace for this thread

# Set thread-specific breakpoints
(gdb) break worker_thread thread 2  # Only break in thread 2

# Control individual threads
(gdb) set scheduler-locking on      # Only current thread runs
(gdb) next                          # Step ONLY this thread
(gdb) set scheduler-locking off     # All threads run again
```

Step 3: Catch a data race:

```
(gdb) watch shared_counter      # Watch the shared variable
(gdb) continue                  # Run until it changes
# Note which thread modified it – is it protected by a lock?
```

Step 4: Answer in `answers.txt` :

1. How many threads does the program create?

2. Is the shared variable properly protected?
3. What happens with `scheduler-locking` on ?

✅ **Checkpoint:** You can debug multi-threaded programs and isolate thread-specific issues.

LAB 2: Tracing with strace and ltrace

Mission: "The System Call Spy"

Story: An application works on the developer's machine but fails in production. No logs, no error messages — just silent failure. You need to spy on every system call and library function the program makes. Enter `strace` and `ltrace` — your x-ray vision into program behavior.

Background

strace traces **system calls** — every interaction with the kernel:

```
Application → glibc → System Call → Kernel
                ↑
            strace hooks here
```

ltrace traces **library calls** — functions in shared libraries:

```
Application → Shared Library (.so)
                ↑
            ltrace hooks here
```

🔧 Exercise 2.1: strace Fundamentals

Objective: Trace system calls to understand program behavior and diagnose failures.

Step 1: Basic tracing:

```
# Trace all system calls of a command
strace ls /tmp

# Trace with timestamps
strace -t ls /tmp          # Wall clock time
strace -T ls /tmp          # Time spent in each syscall
strace -tt ls /tmp         # Microsecond timestamps

# Show string arguments fully (default truncates at 32 chars)
strace -s 256 ls /tmp
```

Step 2: Filter by syscall category:

```
# Only file-related syscalls
strace -e trace=file ls /tmp

# Only network syscalls
strace -e trace=network curl -s http://example.com

# Only process-related
strace -e trace=process bash -c "ls && echo done"

# Only memory-related
```

```
strace -e trace=memory ./my_program

# Specific syscalls
strace -e trace=open,read,write,close cat /etc/hostname

# Negate (everything EXCEPT)
strace -e trace=!write ./my_program
```

Step 3: Trace a running process:

```
# Attach to PID
sudo strace -p $(pgrep nginx) -f

# -f follows forked child processes
# -ff with -o puts each thread/process in separate file
sudo strace -ff -o /tmp/trace -p $(pgrep apache2)
```

Step 4: Save output for analysis:

```
# Write trace to file
strace -o trace.log -s 256 ./my_app

# Count syscalls and show summary
strace -c ./my_app

# Summary sorted by time
strace -c -S time ./my_app
```

Step 5: Diagnose common problems:

```
# "File not found" – where is the program looking?
strace -e trace=file ./broken_app 2>&1 | grep "ENOENT"

# "Permission denied" – which file?
strace -e trace=file ./broken_app 2>&1 | grep "EACCES"

# Slow program – which syscall is slow?
strace -T -c ./slow_app
```

Step 6: Answer in `answers.txt` :

1. How many system calls does `ls /tmp` make?
2. Which syscall opens a file?
3. What does `strace -c` show?

✅ **Checkpoint:** You can trace and filter system calls to diagnose application issues.

🔗 Exercise 2.2: ltrace — Library Call Tracing

Objective: Trace shared library function calls.

Step 1: Basic library tracing:

```
# Trace library calls
ltrace ./my_program
```



```
# With timestamps
ltrace -t ./my_program

# Filter specific libraries
ltrace -e malloc+free ./my_program

# Count library calls
ltrace -c ./my_program
```

Step 2: Compare strace vs ltrace output:

```
# Run both on the same program
strace -o /tmp/strace.log ./my_program
ltrace -o /tmp/ltrace.log ./my_program

# strace shows: open(), read(), write(), mmap() ...
# ltrace shows: printf(), malloc(), fopen(), strlen() ...
```

Step 3: Trace memory allocation patterns:

```
ltrace -e malloc+free+calloc+realloc ./my_program 2>&1 | head -50
# Look for: malloc() calls without matching free() → potential leak
```

Step 4: Answer in `answers.txt` :

1. What is the difference between strace and ltrace?
2. What layer does each tool operate at?
3. How can ltrace help find memory leaks?

✅ **Checkpoint:** You understand when to use strace (kernel boundary) vs ltrace (library boundary).

LAB 3: Address Sanitizer (ASan)

Mission: "The Memory Corruption Detector"

Story: *The team's application passes all tests but occasionally corrupts data in production. Memory corruption bugs are invisible to the naked eye — they silently overwrite memory and manifest as bizarre crashes hours later. The Address Sanitizer is your electron microscope for memory bugs.*

Background

AddressSanitizer (ASan) is a compile-time memory error detector built into GCC and Clang. It catches:

- **Heap buffer overflow** — writing past the end of malloc'd memory
- **Stack buffer overflow** — overflowing stack arrays
- **Use-after-free** — accessing memory after it's been freed
- **Double-free** — freeing memory twice
- **Memory leaks** — allocated memory never freed (with LeakSanitizer)
- **Use-after-return** — using a pointer to a local after the function returns

How it works:

```
| Normal Memory: [ data ] |
```

With ASan:	[redzone][data][redzone]
	↑ ↑
	"poison" "poison"
Any access to redzone → INSTANT ERROR REPORT	

Performance impact: ~2x slower, ~3x more memory. **Never use in production builds** — it's a development tool.

Compile with ASan:

```
gcc -g -fsanitize=address -fno-omit-frame-pointer -o prog prog.c
# For leak detection too:
gcc -g -fsanitize=address,leak -fno-omit-frame-pointer -o prog prog.c
```

❧ Exercise 3.1: Detecting Heap Buffer Overflows

Objective: Use ASan to catch out-of-bounds memory writes.

Step 1: Navigate to the lab directory:

```
cd code/debugging/lab02_sanitizers/
cat heap_overflow.c
```

Step 2: Compile without ASan — observe it appears to "work":

```
gcc -g -o heap_overflow_unsafe heap_overflow.c
./heap_overflow_unsafe
echo "Exit code: $?"      # Probably 0 — bug is SILENT!
```

Step 3: Compile WITH ASan:

```
gcc -g -fsanitize=address -fno-omit-frame-pointer -o heap_overflow heap_overflow.c
./heap_overflow
```

Step 4: Analyze the ASan output: ``

```
==12345==ERROR: AddressSanitizer: heap-buffer-overflow on
address 0x... WRITE of size 4 at 0x... thread T0 #0 0x... in
overflow_function heap_overflow.c:15 #1 0x... in main
heap_overflow.c:25 ... 0x... is located 0 bytes after 40-byte region
[0x..., 0x...) allocated by thread T0 here: #0 0x... in malloc (...) #1
0x... in overflow_function heap_overflow.c:10
```

Key information in the report:

```
- **Type of error:** heap-buffer-overflow
- **Operation:** WRITE of size 4
- **Location:** function name, file, line number
- **Where memory was allocated:** separate stack trace
```

Step 5: Answer in `answers.txt`:

1. What line caused the overflow?
2. How many bytes was the buffer?
3. How many bytes past the end was the write?

 **Checkpoint:** ASan catches buffer overflows that would silently corrupt memory.

🧩 Exercise 3.2: Detecting Use-After-Free

Objective: Catch dangling pointer bugs.

Step 1: Examine and compile the use-after-free example:

```
```bash
cat use_after_free.c
gcc -g -fsanitize=address -fno-omit-frame-pointer -o use_after_free use_after_free.c
./use_after_free
```

#### Step 2: ASan reports:

```
==12345==ERROR: AddressSanitizer: heap-use-after-free on address 0x...
```

```
READ of size 4 at 0x... thread T0
#0 ... in dangling_read use_after_free.c:20
```

freed by thread T0 here:

```
#0 ... in free (...)
#1 ... in dangling_read use_after_free.c:18
```

previously allocated by thread T0 here:

```
#0 ... in malloc (...)
#1 ... in dangling_read use_after_free.c:12
```

Note how ASan shows **three** stack traces: where it was used, where it was freed, and where it was allocated.

#### Step 3: Fix the bug and verify:

```
Edit use_after_free.c to fix the dangling pointer
Rebuild with ASan
gcc -g -fsanitize=address -fno-omit-frame-pointer -o use_after_free use_after_free.c
./use_after_free # Should run clean now
```

 **Checkpoint:** ASan detects use-after-free with full allocation/deallocation history.

---

### 🧩 Exercise 3.3: Stack Buffer Overflow and Memory Leaks

**Objective:** Catch stack-based overflows and leaked memory.

**Step 1:** Stack overflow detection:

```
cat stack_overflow.c
gcc -g -fsanitize=address -fno-omit-frame-pointer -o stack_overflow stack_overflow.c
./stack_overflow
```

ASan catches stack overflows that corrupt adjacent stack variables — a common source of security vulnerabilities.

**Step 2:** Memory leak detection:

```
cat memory_leak.c
gcc -g -fsanitize=address,leak -fno-omit-frame-pointer -o memory_leak memory_leak.c
./memory_leak
```

ASan + LeakSanitizer reports:

```
==12345==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 1024 byte(s) in 1 object(s) allocated from:
 #0 ... in malloc (...)
 #1 ... in leaky_function memory_leak.c:10
 #2 ... in main memory_leak.c:25

SUMMARY: AddressSanitizer: 1024 byte(s) leaked in 1 allocation(s).
```

**Step 3:** ASan environment variables for fine-tuning:

```
Abort on first error (default: continue)
ASAN_OPTIONS=halt_on_error=1 ./program

Disable leak checking
ASAN_OPTIONS=detect_leaks=0 ./program

Detect stack-use-after-return
ASAN_OPTIONS=detect_stack_use_after_return=1 ./program

Increase redzone size
ASAN_OPTIONS=redzone=256 ./program

Combined options
ASAN_OPTIONS=halt_on_error=1:detect_leaks=1:verbosity=1 ./program
```

**Step 4:** Answer in `answers.txt` :

1. What compiler flags enable ASan?
2. What is the performance overhead of ASan?
3. Name 4 types of memory errors ASan detects.
4. How is LeakSanitizer enabled alongside ASan?

✔ **Checkpoint:** You can use ASan to catch a wide variety of memory corruption bugs at development time.

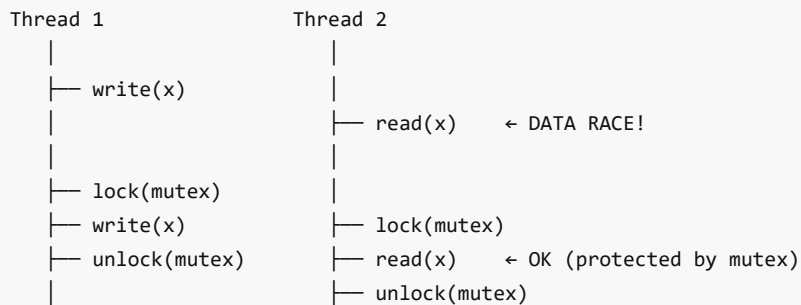
## LAB 4: Thread Sanitizer (TSan)

 **Mission: "The Race Condition Radar"**

**Story:** The multi-threaded server works perfectly under low load. Under heavy load, it produces garbled output. Sometimes. The developers suspect a "timing issue" but can't reproduce it. Thread Sanitizer sees what humans cannot — it detects data races mathematically, regardless of timing.

## Background

**ThreadSanitizer (TSan)** detects data races — when two threads access the same memory location concurrently and at least one access is a write, without proper synchronization.



Compile with TSan:

```
gcc -g -fsanitize=thread -fno-omit-frame-pointer -o prog prog.c -lpthread
```

⚠️ **TSan and ASan cannot be used together** — they use incompatible memory layouts. Use them separately.

## Exercise 4.1: Detecting Data Races

**Objective:** Find and fix data races using TSan.

**Step 1:** Navigate to the lab directory:

```
cd code/debugging/lab02_sanitizers/
cat data_race.c
```

**Step 2:** Compile with TSan:

```
gcc -g -fsanitize=thread -fno-omit-frame-pointer -o data_race data_race.c -lpthread
./data_race
```

## Step 3: Analyze TSan output: ``

WARNING: ThreadSanitizer: data race (pid=12345) Write of size 4 at 0x... by thread T2: #0 increment data\_race.c:15

Previous read of size 4 at 0x... by thread T1: #0 increment data\_race.c:14

Location is global 'shared\_counter' of size 4 at 0x...

Thread T1 (tid=..., running) created by main thread at: #0 pthread\_create (...) #1 main data\_race.c:30

**Thread T2 (tid=..., running) created by main thread at: #0  
pthread\_create (...) #1 main data\_race.c:31**

TSan shows:

- **What** was accessed (variable name, size)
- **Which threads** were racing
- **Where** each thread was created
- **Stack traces** for both accesses

**Step 4:** Fix the race with proper synchronization and verify:

```
```bash
```

After adding mutex protection:

```
gcc -g -fsanitize=thread -fno-omit-frame-pointer -o data_race data_race.c -lpthread
./data_race      # Should run clean – no warnings
```

Step 5: Test with more complex scenarios:

```
# Lock-order inversions (potential deadlocks)
TSAN_OPTIONS=detect_deadlocks=1 ./data_race

# Suppress known issues
echo "race:known_library_function" > tsan_suppressions.txt
TSAN_OPTIONS=suppressions=tsan_suppressions.txt ./data_race
```

Step 6: Answer in `answers.txt` :

1. What is a data race?
2. Why can't TSan and ASan be used together?
3. What synchronization primitive fixed the race?

✅ **Checkpoint:** You can detect data races at compile time using TSan.

LAB 5: Valgrind

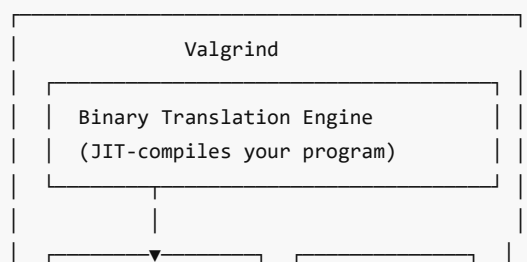
Mission: "The Memory Detective"

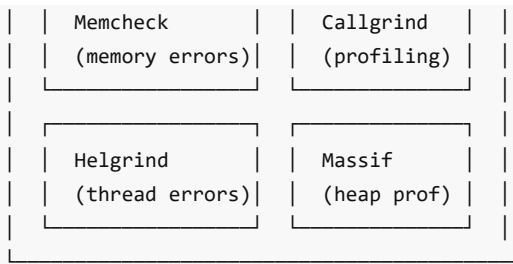
Story: The QA team reports that the application's memory usage grows steadily until it exhausts all system RAM after running for 48 hours. Classic memory leak. Valgrind's Memcheck tool will find every leaked byte and tell you exactly where it was allocated.

Background

Valgrind is a dynamic analysis framework. Its most famous tool is **Memcheck**, which detects:

- Memory leaks (lost blocks)
- Use of uninitialized memory
- Invalid reads/writes (buffer overflows)
- Double-free / mismatched free
- Overlapping memcpy





Performance impact: ~20-30x slower. Compile with `-g` for source-level info.

❏ Exercise 5.1: Memcheck — Finding Memory Leaks

Objective: Use Valgrind Memcheck to find and fix memory leaks.

Step 1: Navigate to the lab directory:

```
cd code/debugging/lab03_valgrind/
cat leak_demo.c
```

Step 2: Compile with debug symbols (no sanitizers — Valgrind has its own instrumentation):

```
gcc -g -O0 -o leak_demo leak_demo.c
```

Step 3: Run under Valgrind:

```
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./leak_demo
```

Step 4: Understand the output:

```
==12345== HEAP SUMMARY:
==12345==      in use at exit: 2,048 bytes in 3 blocks
==12345==    total heap usage: 10 allocs, 7 frees, 4,096 bytes allocated
==12345==
==12345== 1,024 bytes in 1 blocks are definitely lost in loss record 1 of 3
==12345==    at 0x...: malloc (vg_replace_malloc.c:...)
==12345==    by 0x...: leaky_function (leak_demo.c:15)
==12345==    by 0x...: main (leak_demo.c:40)
==12345==
==12345== 512 bytes in 1 blocks are indirectly lost in loss record 2 of 3
==12345==    ...
==12345==
==12345== LEAK SUMMARY:
==12345==    definitely lost: 1,024 bytes in 1 blocks
==12345==    indirectly lost: 512 bytes in 1 blocks
==12345==    possibly lost: 0 bytes in 0 blocks
==12345==    still reachable: 512 bytes in 1 blocks
==12345==    suppressed: 0 bytes in 0 blocks
```

Leak categories:

- **Definitely lost** — memory with no pointers to it. Fix these first!
- **Indirectly lost** — memory reachable only through "definitely lost" blocks
- **Possibly lost** — ambiguous — could be a leak or an interior pointer
- **Still reachable** — memory still pointed to at exit (often globals/singletons)

Step 5: Fix all leaks and verify:

```
# After fixing:
valgrind --leak-check=full ./leak_demo
# Target: "All heap blocks were freed -- no leaks are possible"
```

✅ **Checkpoint:** You can find and categorize memory leaks using Valgrind.

❏ Exercise 5.2: Memcheck — Invalid Memory Access

Objective: Detect out-of-bounds access and use of uninitialized variables.

Step 1: Run the invalid access demo:

```
cat invalid_access.c
gcc -g -O0 -o invalid_access invalid_access.c
valgrind --track-origins=yes ./invalid_access
```

Step 2: Analyze different error types:

```
# Invalid read
==12345== Invalid read of size 4
==12345==    at 0x...: read_past_end (invalid_access.c:10)
==12345==   Address 0x... is 0 bytes after a block of size 40 alloc'd

# Uninitialized value
==12345== Conditional jump or move depends on uninitialised value(s)
==12345==    at 0x...: use_uninit (invalid_access.c:20)
==12345==   Uninitialised value was created by a heap allocation
==12345==    at 0x...: malloc (...)
==12345==    by 0x...: use_uninit (invalid_access.c:18)
```

Step 3: Use Valgrind with additional tools:

```
# Callgrind – CPU profiling
valgrind --tool=callgrind ./program
callgrind_annotate callgrind.out.*

# Helgrind – thread error detection
valgrind --tool=helgrind ./threaded_program

# Massif – heap profiler
valgrind --tool=massif ./program
ms_print massif.out.*
```

Step 4: Generate Valgrind suppressions for known issues:

```
# Generate suppression entries
valgrind --gen-suppressions=all ./program 2> suppressions.raw

# Use suppressions file
valgrind --suppressions=my_suppressions.supp ./program
```

Step 5: Answer in `answers.txt` :

1. What is the performance overhead of Valgrind?
2. What does `--track-origins=yes` do?
3. What is the difference between "definitely lost" and "still reachable"?
4. Name 3 Valgrind tools besides Memcheck.

✅ **Checkpoint:** You can use Valgrind to detect memory access errors and leaks comprehensively.

LAB 6: Segfault Debugging

Mission: "Crash Scene Investigator"

Story: The production server crashed at 3 AM. A core dump was saved. Management wants a root cause analysis by morning. You must reconstruct the crash from the evidence — the core dump, signal information, and stack trace. Every segfault tells a story if you know how to read it.

Background

A **segmentation fault** (SIGSEGV) occurs when a process accesses memory it doesn't have permission to access:

- Dereferencing a NULL pointer
- Writing to read-only memory
- Accessing freed memory (use-after-free)
- Stack overflow
- Buffer overflow corrupting return addresses

Segfault Analysis Workflow

1. Enable core dumps (`ulimit -c unlimited`)
2. Reproduce the crash
3. Load core in GDB
4. `backtrace` → locate crash point
5. Examine variables & memory
6. Check signal info (`info signal`)
7. Review memory map (`/proc/PID/maps`)
8. Identify root cause & fix

🔧 Exercise 6.1: Systematic Segfault Analysis

Objective: Develop a systematic workflow for diagnosing segfaults.

Step 1: Navigate to the lab directory:

```
cd code/debugging/lab01_gdb/
cat crash_scenarios.c
```

This program contains multiple intentional crash scenarios.

Step 2: Enable core dumps and run:

```
ulimit -c unlimited
make
./crash_scenarios 1    # Scenario 1: NULL pointer
./crash_scenarios 2    # Scenario 2: Use-after-free
```

```
./crash_scenarios 3    # Scenario 3: Stack overflow
./crash_scenarios 4    # Scenario 4: Double-free
```

Step 3: For each crash, perform full analysis:

```
gdb ./crash_scenarios core.*

(gdb) bt full                # Full backtrace with locals
(gdb) info signal SIGSEGV    # Signal details
(gdb) frame 0                # Go to crash frame
(gdb) info locals            # Local variables
(gdb) info registers         # Register state
(gdb) x/i $pc                # Instruction that faulted
(gdb) info proc mappings     # Memory map
```

Step 4: Use dmesg to find kernel-side crash info:

```
dmesg | tail -20
# Look for: "segfault at <addr> ip <addr> sp <addr> error <code>"
# Error code bits:
#   bit 0: 0=no-page-found, 1=protection-violation
#   bit 1: 0=read, 1=write
#   bit 2: 0=kernel-mode, 1=user-mode
```

Step 5: Document each crash in `answers.txt` : For each scenario, write:

1. Crash address
2. Faulting instruction
3. Root cause
4. Fix

✅ **Checkpoint:** You have a systematic methodology for analyzing any segfault.

❧ Exercise 6.2: Advanced Crash Analysis Techniques

Objective: Handle complex crash scenarios including corrupted stacks.

Step 1: When the stack is corrupted:

```
# Normal backtrace may fail:
(gdb) bt
# ... <corrupted>

# Try manual stack walking:
(gdb) info registers        # Get RSP (stack pointer)
(gdb) x/50gx $rsp           # Examine raw stack contents
(gdb) info symbol 0x7fff...  # Identify addresses on stack

# Use frame pointer chain (if compiled with -fno-omit-frame-pointer):
(gdb) x/2gx $rbp            # [saved_rbp, return_address]
```

Step 2: Analyze address patterns:

```
Address 0x0000000000000000 → NULL pointer dereference
Address 0x0000000000000008 → NULL struct pointer + offset 8
```

Address 0x4141414141414141 → Buffer overflow (0x41 = 'A')
Address 0x00007f... → Likely valid user-space address
Address 0xffff... → Kernel address (shouldn't appear in user crash)
Address 0xdeadbeef → Uninitialized memory marker

Step 3: Use addr2line for quick symbol lookup:

```
# Convert address to file:line
addr2line -e ./crash_scenarios 0x401234

# With function name
addr2line -f -e ./crash_scenarios 0x401234

# Demangle C++ names
addr2line -f -C -e ./crash_scenarios 0x401234
```

✔ **Checkpoint:** You can analyze even complex crashes with corrupted stacks.

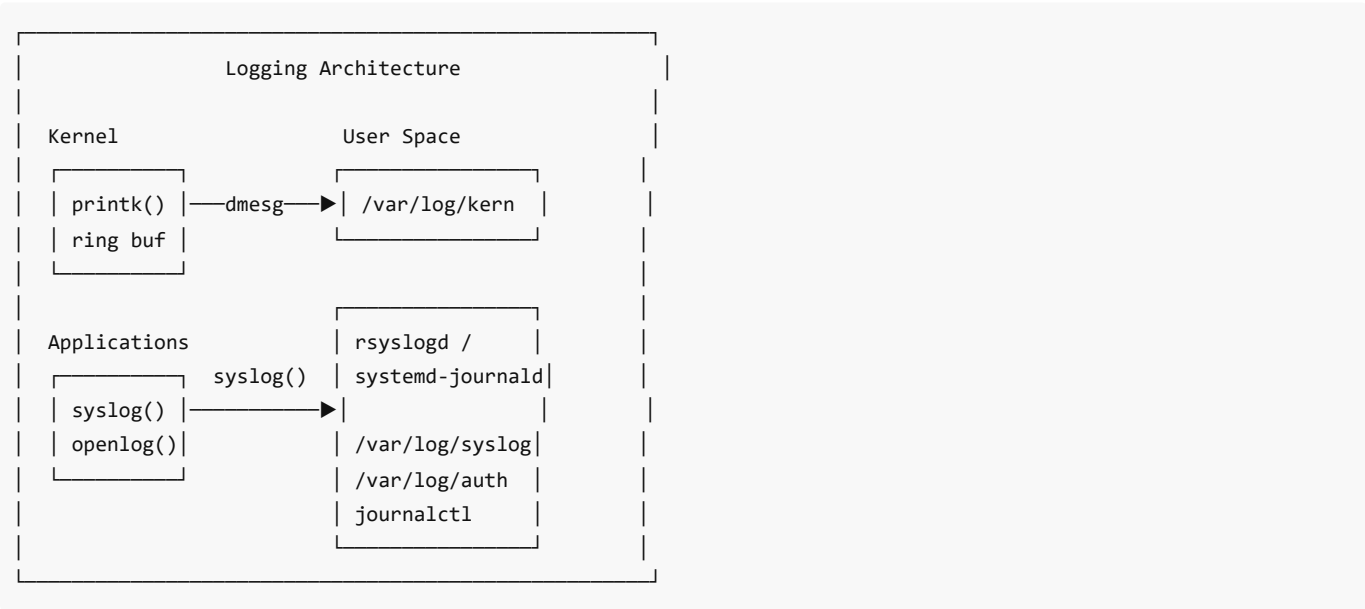
LAB 7: syslog and Kernel Logging

🎯 Mission: "The Log Whisperer"

Story: In a production fleet of 500 servers, problems are diagnosed from logs — not by connecting GDB to each machine. Understanding the logging infrastructure — syslog, journald, dmesg — is what separates a developer from a systems engineer. Master the logs, master the system.

📖 Background

Linux has multiple logging systems:



Syslog severity levels:

Level	Name	Description
0	EMERG	System is unusable
1	ALERT	Action must be taken immediately

2	CRIT	Critical conditions
3	ERR	Error conditions
4	WARNING	Warning conditions
5	NOTICE	Normal but significant
6	INFO	Informational
7	DEBUG	Debug-level messages

🔗 Exercise 7.1: Exploring System Logs

Objective: Navigate the Linux logging infrastructure.

Step 1: Explore log files:

```
# Traditional syslog files
ls -la /var/log/
cat /var/log/syslog | tail -20
cat /var/log/auth.log | tail -10
cat /var/log/kern.log | tail -10

# Kernel ring buffer
dmesg | tail -30
dmesg -T | tail -30           # Human-readable timestamps
dmesg -H | tail -30           # Colorized, pager
dmesg --level=err,warn        # Only errors and warnings
```

Step 2: journalctl — systemd journal:

```
# Recent entries
journalctl -n 50              # Last 50 entries
journalctl -f                 # Follow (like tail -f)

# Filter by unit
journalctl -u sshd            # SSH daemon logs
journalctl -u nginx           # Nginx logs

# Filter by time
journalctl --since "2024-01-01 12:00"
journalctl --since "1 hour ago"
journalctl --since today

# Filter by priority
journalctl -p err              # Errors and above
journalctl -p warning..err     # Range

# Filter by boot
journalctl -b                  # Current boot
journalctl -b -1               # Previous boot
journalctl --list-boots        # All boots

# Kernel messages only
journalctl -k                  # Same as dmesg but persistent
```

```
# Output formats
journalctl -o json-pretty      # JSON format
journalctl -o verbose         # All fields
```

Step 3: Use the syslog API from C:

```
cd code/debugging/lab01_gdb/      # Use any lab directory
cat > syslog_demo.c << 'EOF'
#include <syslog.h>
#include <stdio.h>

int main(void)
{
    /* Open connection to syslog */
    openlog("my_app", LOG_PID | LOG_CONS, LOG_USER);

    /* Log at different levels */
    syslog(LOG_INFO,    "Application started");
    syslog(LOG_WARNING, "Configuration file missing, using defaults");
    syslog(LOG_ERR,     "Failed to connect to database: %s", "timeout");
    syslog(LOG_DEBUG,   "Debug: processed %d items", 42);

    /* Close connection */
    closelog();

    printf("Messages sent to syslog. Check with:\n");
    printf("  journalctl -t my_app\n");
    printf("  grep my_app /var/log/syslog\n");
    return 0;
}
EOF
gcc -g -o syslog_demo syslog_demo.c
./syslog_demo
journalctl -t my_app --no-pager
```

Step 4: Answer in `answers.txt` :

1. What are the 8 syslog severity levels?
2. How do you filter journalctl by service and time?
3. What is the difference between `/var/log/syslog` and `journalctl` ?

✅ **Checkpoint:** You can navigate and query the Linux logging infrastructure.

PART 2 — KERNEL DEBUGGING

LAB 8: Ftrace — Function Tracer

Mission: "The Kernel Function Tracker"

Story: A kernel module is causing intermittent system slowdowns. You need to trace which kernel functions are being called, how long they take, and what code paths the kernel follows. Ftrace is the kernel's built-in flight recorder — always ready, minimal overhead.

Background

Ftrace is the Linux kernel's built-in tracing framework:

- **Function tracer** — trace every kernel function call
- **Function graph tracer** — trace function entry/exit with call graph
- **Event tracing** — trace specific kernel events (scheduler, IRQ, etc.)
- **Latency tracers** — find preemption/IRQ latency

Ftrace Architecture	
/sys/kernel/tracing/ (or /sys/kernel/debug/tracing/)	
— current_tracer	← Which tracer is active
— trace	← Read trace output
— trace_pipe	← Streaming trace output
— tracing_on	← Enable/disable (1/0)
— set_ftrace_filter	← Filter by function name
— set_graph_function	← Function graph filter
— available_tracers	← List available tracers
— available_filter_functions	← Traceable functions
— events/	← Event tracing directory

🔗 Exercise 8.1: Basic Function Tracing

Objective: Trace kernel functions using the ftrace interface. 🍓

Step 1: Check ftrace availability:

```
# Mount tracefs if not already mounted
sudo mount -t tracefs nodev /sys/kernel/tracing 2>/dev/null

# Check available tracers
cat /sys/kernel/tracing/available_tracers

# Check current tracer
cat /sys/kernel/tracing/current_tracer
```

Step 2: Enable function tracing:

```
# Set the tracer
echo function | sudo tee /sys/kernel/tracing/current_tracer

# Enable tracing
echo 1 | sudo tee /sys/kernel/tracing/tracing_on

# Generate some kernel activity
ls /tmp > /dev/null
cat /proc/cpuinfo > /dev/null

# Read the trace
sudo cat /sys/kernel/tracing/trace | head -50
```

```
# Disable tracing
echo 0 | sudo tee /sys/kernel/tracing/tracing_on
```

Step 3: Filter to specific functions:

```
# Clear the filter
echo | sudo tee /sys/kernel/tracing/set_ftrace_filter

# Trace only VFS functions
echo 'vfs_*' | sudo tee /sys/kernel/tracing/set_ftrace_filter

# Verify filter
cat /sys/kernel/tracing/set_ftrace_filter

# Enable and trigger
echo 1 | sudo tee /sys/kernel/tracing/tracing_on
cat /etc/hostname > /dev/null
echo 0 | sudo tee /sys/kernel/tracing/tracing_on

# Read filtered trace
sudo cat /sys/kernel/tracing/trace | head -30
```

Step 4: Function graph tracer (shows call hierarchy):

```
# Set function_graph tracer
echo function_graph | sudo tee /sys/kernel/tracing/current_tracer

# Filter to a specific function and its callees
echo do_sys_open | sudo tee /sys/kernel/tracing/set_graph_function

# Trace
echo 1 | sudo tee /sys/kernel/tracing/tracing_on
cat /etc/hostname > /dev/null
echo 0 | sudo tee /sys/kernel/tracing/tracing_on

# Beautiful call graph output:
sudo cat /sys/kernel/tracing/trace | head -60
```

The function graph tracer output looks like:

```
# DURATION      | FUNCTION CALLS
# -----      | -----
0.550 us       | do_sys_open() {
0.120 us       |     getname();
0.080 us       |     get_unused_fd_flags();
               |     do_filp_open() {
0.090 us       |         path_openat() {
0.060 us       |             link_path_walk();
...           |
```

Step 5: Clean up:

```
echo nop | sudo tee /sys/kernel/tracing/current_tracer
echo | sudo tee /sys/kernel/tracing/set_ftrace_filter
```

```
echo | sudo tee /sys/kernel/tracing/set_graph_function
```

✅ **Checkpoint:** You can trace kernel functions and generate call graphs.

🔗 Exercise 8.2: Event Tracing and trace-cmd

Objective: Use kernel event tracing and the trace-cmd tool. 🍓

Step 1: Explore available events:

```
# List all event categories
ls /sys/kernel/tracing/events/

# List scheduler events
ls /sys/kernel/tracing/events/sched/

# View event format
cat /sys/kernel/tracing/events/sched/sched_switch/format
```

Step 2: Enable specific events:

```
# Enable scheduler switch events
echo 1 | sudo tee /sys/kernel/tracing/events/sched/sched_switch/enable

# Enable IRQ events
echo 1 | sudo tee /sys/kernel/tracing/events/irq/irq_handler_entry/enable

# Trace
echo 1 | sudo tee /sys/kernel/tracing/tracing_on
sleep 1
echo 0 | sudo tee /sys/kernel/tracing/tracing_on

# Read
sudo cat /sys/kernel/tracing/trace | head -40

# Disable
echo 0 | sudo tee /sys/kernel/tracing/events/sched/sched_switch/enable
echo 0 | sudo tee /sys/kernel/tracing/events/irq/irq_handler_entry/enable
```

Step 3: Use trace-cmd (user-friendly ftrace frontend):

```
# Install trace-cmd
sudo apt install -y trace-cmd

# Record scheduler events for 5 seconds
sudo trace-cmd record -e sched_switch -e sched_wakeup sleep 5

# View the recording
trace-cmd report | head -50

# Record function graph for a specific command
sudo trace-cmd record -p function_graph -g do_sys_open cat /etc/hostname
trace-cmd report | head -40

# Record with filtering
```



```
sudo trace-cmd record -p function -l 'vfs_*' cat /etc/hostname
trace-cmd report | head -30
```

Step 4: Visualize with KernelShark:

```
# Install KernelShark (GUI)
sudo apt install -y kernelshark

# Record and visualize
sudo trace-cmd record -e sched_switch sleep 5
kernelshark trace.dat      # Opens GUI visualizer
```

Step 5: Answer in `answers.txt` :

1. What is the difference between function tracing and event tracing?
2. How do you filter ftrace to specific functions?
3. What does the function graph tracer show that function tracer doesn't?

✅ **Checkpoint:** You can use ftrace and trace-cmd to trace kernel internals.

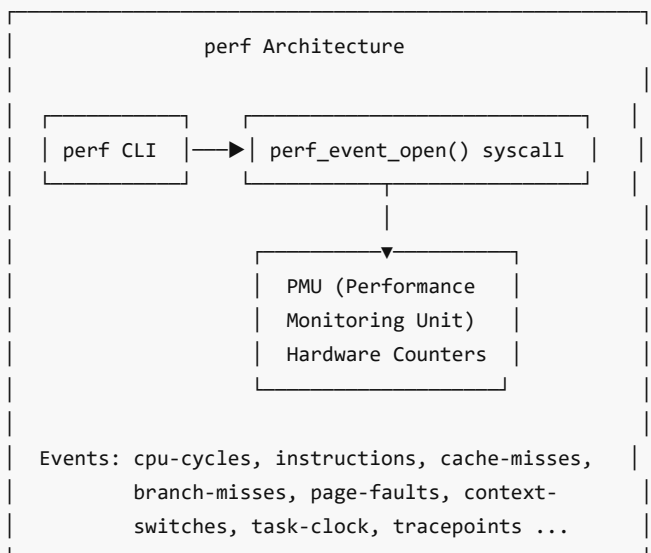
LAB 9: Perf — Performance Analysis

🎯 Mission: "The Performance Profiler"

Story: The server's CPU usage spiked to 100%. Is it user code? Kernel code? Cache misses? Branch mispredictions? Perf gives you access to the CPU's hardware performance counters — the truth that no software can hide.

📖 Background

perf is Linux's primary performance analysis tool. It uses hardware performance counters and kernel tracepoints.



🔧 Exercise 9.1: Basic Performance Profiling

Objective: Profile CPU-intensive applications with perf. 🍓

Step 1: Check perf availability:

```
perf --version

# List available events
perf list | head -40

# Hardware events
perf list hw

# Software events
perf list sw
```

Step 2: Count events:

```
# Count basic stats for a command
perf stat ls /tmp

# Detailed stats
perf stat -d ./my_program

# Specific events
perf stat -e cycles,instructions,cache-misses,branch-misses ./my_program

# Repeat for statistical accuracy
perf stat -r 10 ./my_program      # Run 10 times, show averages
```

Step 3: Record and report CPU profiles:

```
# Record samples (default: CPU cycles)
sudo perf record -g ./my_program

# Report interactively
sudo perf report

# Report to stdout
sudo perf report --stdio | head -50

# Record a running process
sudo perf record -g -p $(pgrep my_program) sleep 10

# Record system-wide
sudo perf record -g -a sleep 5
```

Step 4: Generate flame graphs:

```
# Install FlameGraph tools
git clone https://github.com/brendangregg/FlameGraph.git

# Record and generate
sudo perf record -g -F 99 ./my_program
sudo perf script | ./FlameGraph/stackcollapse-perf.pl | \
  ./FlameGraph/flamegraph.pl > flamegraph.svg
```

```
# Open in browser
# firefox flamegraph.svg
```

Step 5: Top-like live profiling:

```
# Live function-level profiling
sudo perf top

# Filter to specific process
sudo perf top -p $(pgrep my_program)

# Show only kernel functions
sudo perf top -K
```

Step 6: Answer in `answers.txt` :

1. What are hardware performance counters?
2. What is the IPC (instructions per cycle) of your test program?
3. What does a flame graph show?

✓ **Checkpoint:** You can profile applications for CPU bottlenecks.

🔗 Exercise 9.2: Perf for Kernel Analysis

Objective: Use perf to analyze kernel performance and scheduler behavior.

Step 1: Trace scheduler events:

```
# Record scheduler events
sudo perf sched record sleep 5
sudo perf sched latency      # Scheduling latency
sudo perf sched map         # Thread-to-CPU mapping over time
```

Step 2: Analyze cache behavior:

```
# Cache miss analysis
sudo perf stat -e L1-dcache-loads,L1-dcache-load-misses,LLC-loads,LLC-load-misses ./my_program

# Calculate miss rates
# L1 miss rate = L1-dcache-load-misses / L1-dcache-loads
# LLC miss rate = LLC-load-misses / LLC-loads
```

Step 3: Lock contention analysis:

```
# Record lock events
sudo perf lock record ./threaded_program
sudo perf lock report

# Contention stats
sudo perf lock contention
```

Step 4: Memory access patterns:

```
# Record memory access events (requires hardware support)
sudo perf mem record ./my_program
```

```
sudo perf mem report
```

✅ **Checkpoint:** You can analyze kernel scheduling, cache efficiency, and lock contention.

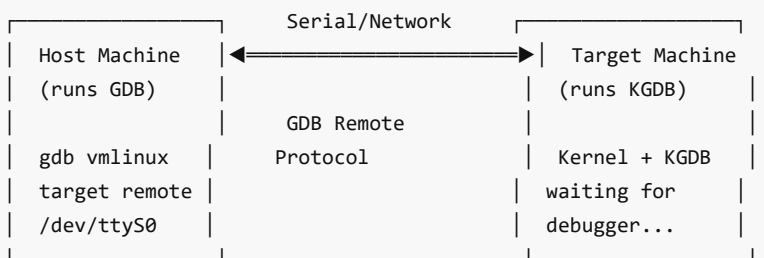
LAB 10: KGDB — Kernel Debugger

🎯 Mission: "Debugging the Kernel Itself"

Story: A kernel module is causing a panic. You can see the crash trace in `dmesg`, but you need to step through the kernel code instruction by instruction. KGDB connects GDB to the running kernel — you're now debugging the operating system itself.

📖 Background

KGDB is a kernel debugger that uses GDB's remote debugging protocol. You connect from a second machine (or a serial console) to debug the kernel.



⚠️ **KGDB halts the entire kernel** on the target machine. Use only in VMs or with serial console access.

🔧 Exercise 10.1: Setting Up KGDB

Objective: Configure a kernel for KGDB debugging.

Step 1: Check if KGDB is enabled:

```
# Check kernel config
cat /boot/config-$(uname -r) | grep -i kgdb
# CONFIG_KGDB=y
# CONFIG_KGDB_SERIAL_CONSOLE=y
# CONFIG_KGDB_KDB=y
```

Step 2: Boot with KGDB parameters (add to GRUB):

```
# Edit GRUB configuration
sudo nano /etc/default/grub

# Add to GRUB_CMDLINE_LINUX:
GRUB_CMDLINE_LINUX="kgdboc=ttyS0,115200 kgdbwait"
# kgdboc: KGDB over console (serial port)
# kgdbwait: Wait for debugger at boot

# Update GRUB
sudo update-grub
```

Step 3: For QEMU/VM testing (recommended):

```
# Launch QEMU with serial port for KGDB
qemu-system-x86_64 \
  -kernel /boot/vmlinuz-$(uname -r) \
  -initrd /boot/initrd.img-$(uname -r) \
  -append "root=/dev/sda1 kgdboc=ttyS0,115200 kgdbwait nokaslr" \
  -serial tcp::1234,server,nowait \
  -hda disk.img \
  -m 1G

# On host, connect GDB:
gdb ./vmlinux
(gdb) target remote localhost:1234
(gdb) continue
```

Step 4: Trigger KGDB manually on a running system:

```
# Enter the debugger from a running kernel
echo g | sudo tee /proc/sysrq-trigger

# Or with Magic SysRq key: Alt+SysRq+G
```

Step 5: Essential KGDB/GDB commands:

```
# Connect
(gdb) target remote /dev/ttyS0

# Kernel-specific inspection
(gdb) lx-dmesg           # View kernel log
(gdb) lx-ps             # List processes
(gdb) lx-lsmod           # List modules

# Set breakpoints in kernel functions
(gdb) break do_sys_open
(gdb) break my_module.c:42

# Inspect kernel data structures
(gdb) print init_task    # PID 0 task_struct
(gdb) print *current     # Current task
```

Step 6: Answer in `answers.txt` :

1. What is the purpose of `kgdbwait` ?
2. Why is `nokaslr` needed for debugging?
3. What is the difference between KGDB and KDB?

✓ **Checkpoint:** You understand KGDB setup and can connect GDB to a kernel.

LAB 11: KDB — Kernel Built-in Debugger

Mission: "Emergency Kernel Shell"

Story: The system hangs. No SSH, no GDB connection. But the serial console is still alive. KDB gives you a debugger shell directly on the crashed kernel — no external tools required. It's the kernel's emergency room.

Background

KDB is an in-kernel debugger — it runs inside the kernel itself (no external host needed). It provides a command-line interface on the console.

KDB vs KGDB:

KGDB	KDB
External	Built-in
GDB client	shell
Full source	Limited
debugging	commands
Needs host	No host
machine	needed

❏ Exercise 11.1: Using KDB

Objective: Use KDB for emergency kernel debugging.

Step 1: Enable KDB:

```
# Check if KDB is available
cat /boot/config-$(uname -r) | grep KDB

# Boot with kdb on console
# GRUB: kgdboc=kbd      (keyboard)
# or:   kgdboc=ttyS0    (serial)
```

Step 2: Enter KDB:

```
# Via SysRq
echo g | sudo tee /proc/sysrq-trigger

# KDB prompt appears:
# kdb>
```

Step 3: Essential KDB commands:

```
kdb> help          # List all commands
kdb> ps            # Process list
kdb> btp <pid>     # Backtrace for process
kdb> bt           # Backtrace current context
kdb> lsmod         # Loaded modules
kdb> dmesg         # Kernel log
kdb> md <addr>     # Memory dump
kdb> go           # Continue execution
kdb> sr h          # SysRq help
kdb> sr b          # SysRq reboot
kdb> rd           # Display registers
kdb> bp <addr>     # Set breakpoint
kdb> bc <num>     # Clear breakpoint
kdb> bl           # List breakpoints
kdb> ss           # Single step
```

Step 4: Practice diagnosing a stuck system:

```
kdb> ps                # Who's running? Who's stuck?
kdb> btp 1             # Backtrace of init (PID 1)
kdb> dmesg             # Any errors in the log?
kdb> lsmod             # Suspicious modules?
kdb> go                # Resume the kernel
```

Step 5: Answer in `answers.txt` :

1. When would you use KDB instead of KGDB?
2. How do you enter KDB on a running system?
3. What does `btp` show?

✅ **Checkpoint:** You can use KDB as an emergency debugger when no external tools are available.

LAB 12: LTTng — Linux Trace Toolkit Next Generation

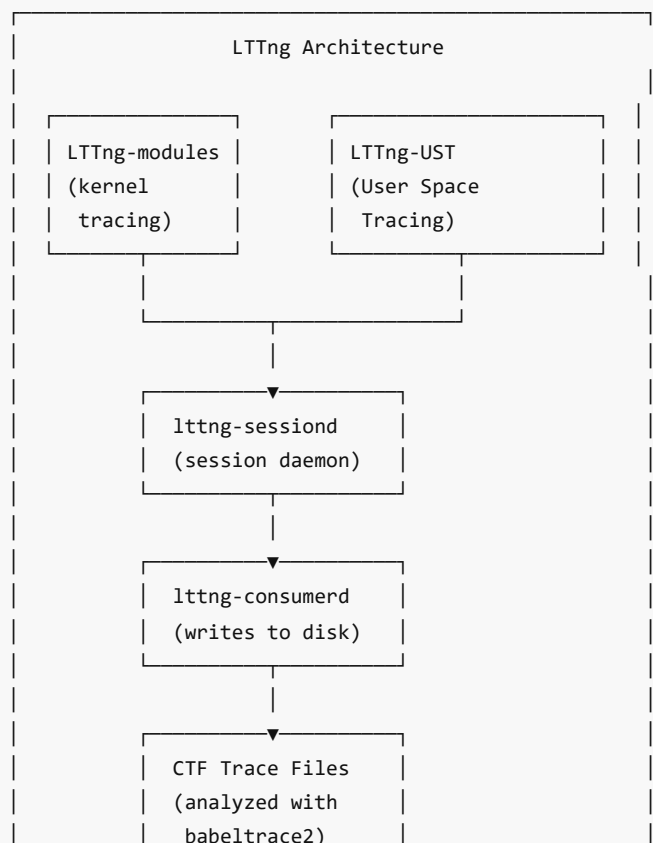
🚀 Mission: "The High-Speed Flight Recorder"

Story: The system exhibits performance anomalies that last only a few microseconds — too fast for `printk`, too frequent for GDB breakpoints. LTTng records billions of events with near-zero overhead, giving you a microscopic view of both kernel and user space.

📖 Background

LTTng is a high-performance tracing framework designed for production use:

- **Near-zero overhead** — uses per-CPU ring buffers and lock-free algorithms
- **Kernel and user space tracing** — unified view of the whole system
- **Binary trace format (CTF)** — compact and fast to read
- **Flight recorder mode** — overwrites oldest events, always ready



❏ Exercise 12.1: LTTng Kernel Tracing

Objective: Set up and use LTTng for kernel event tracing.

Step 1: Install LTTng:

```
sudo apt install -y lttng-tools lttng-modules-dkms babeltrace2

# Verify
lttng --version

# Start the session daemon
lttng-sessiond --daemonize
```

Step 2: Create a tracing session:

```
# Create session
lttng create my-kernel-session

# List available kernel events
lttng list --kernel | head -50

# Enable scheduler events
lttng enable-event --kernel sched_switch,sched_wakeup

# Enable syscall events
lttng enable-event --kernel --syscall open,read,write,close

# Start tracing
lttng start

# Do something interesting
ls /tmp > /dev/null
cat /etc/hostname > /dev/null
sleep 2

# Stop and view
lttng stop
lttng view    # Uses babeltrace2

# Or with babeltrace2 directly
babeltrace2 ~/lttng-traces/my-kernel-session-* | head -50

# Destroy session
lttng destroy
```

Step 3: Flight recorder (snapshot) mode:

```
# Create snapshot session – always recording, dump on demand
lttng create my-flight-recorder --snapshot

lttng enable-event --kernel sched_switch,irq_handler_entry
```



```
lttng start

# ... system runs normally ...

# Something weird happens – grab snapshot:
lttng snapshot record

lttng stop
babeltrace2 ~/lttng-traces/my-flight-recorder-* | head -30

lttng destroy
```

Step 4: Analyze with babeltrace2:

```
# Filter events
babeltrace2 ~/lttng-traces/my-session-* | grep sched_switch | head -20

# Convert to text
babeltrace2 ~/lttng-traces/my-session-* --output-format=text > trace.txt

# Statistics
babeltrace2 ~/lttng-traces/my-session-* | wc -l    # Total events
```

Step 5: Answer in `answers.txt` :

1. What makes LTTng different from ftrace?
2. What is flight recorder mode?
3. What format does LTTng use for trace files?

✅ **Checkpoint:** You can use LTTng for high-performance kernel tracing.

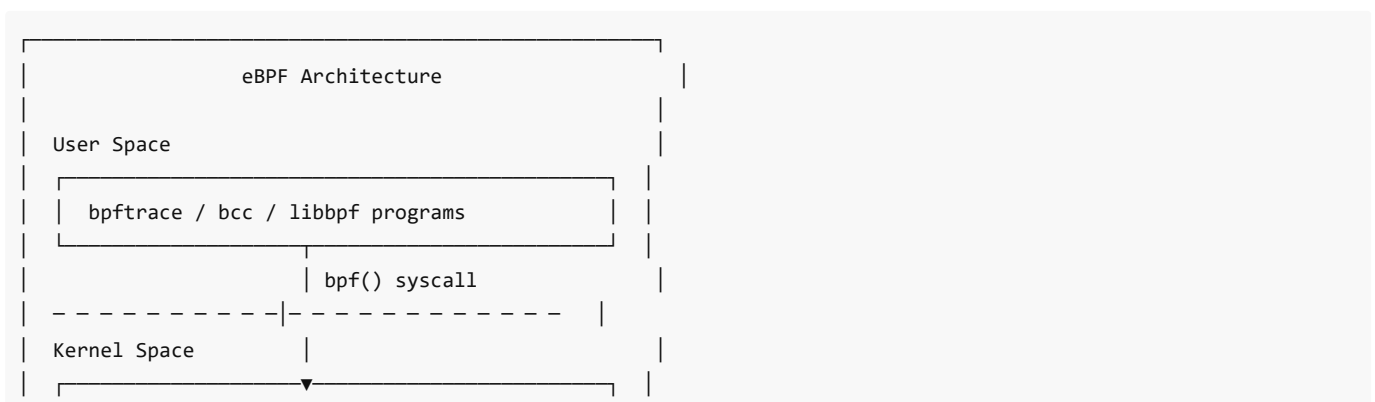
LAB 13: eBPF — Extended Berkeley Packet Filter

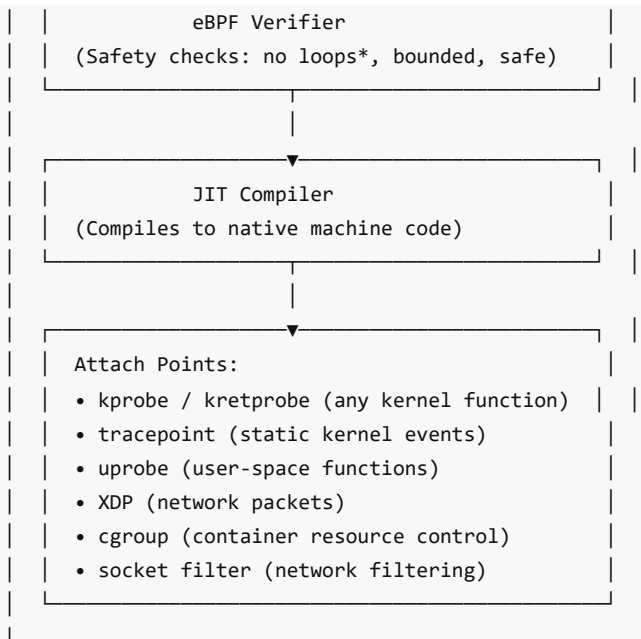
🎯 Mission: "The Programmable Kernel Observer"

Story: Traditional tracing tools show you what the kernel developers decided to instrument. eBPF lets you write your own programs that run safely inside the kernel — attach them to any function, tracepoint, or network event. It's like having programmable x-ray vision into the kernel.

📖 Background

eBPF is a revolutionary technology that lets you run sandboxed programs in the Linux kernel:





❏ Exercise 13.1: bpftrace — One-liners for Kernel Tracing

Objective: Use bpftrace for quick kernel analysis. 🍓

Step 1: Install bpftrace:

```
sudo apt install -y bpftrace
```

Step 2: Essential one-liners:

```
# List all tracepoints
sudo bpftrace -l 'tracepoint:*' | head -20

# List kprobes (kernel functions)
sudo bpftrace -l 'kprobe:vfs_*'

# Hello World – trace sys_enter
sudo bpftrace -e 'tracepoint:raw_syscalls:sys_enter { @[comm] = count(); }' &
sleep 5; kill %1

# Trace file opens with filename
sudo bpftrace -e '
tracepoint:syscalls:sys_enter_openat {
    printf("%s opened %s\n", comm, str(args.filename));
}'

# Trace process creation
sudo bpftrace -e '
tracepoint:sched:sched_process_exec {
    printf("exec: %s (pid=%d)\n", comm, pid);
}'

# Histogram of read() sizes
sudo bpftrace -e '
tracepoint:syscalls:sys_exit_read /args.ret > 0/ {
    @bytes = hist(args.ret);
}
```

```

}'

# Trace block I/O latency
sudo bpftrace -e '
kprobe:blk_account_io_start { @start[arg0] = nsecs; }
kprobe:blk_account_io_done /@start[arg0]/ {
    @usecs = hist((nsecs - @start[arg0]) / 1000);
    delete(@start[arg0]);
}'

```

Step 3: Write a multi-line bpftrace script:

```

cat > /tmp/syscall_monitor.bt << 'EOF'
#!/usr/bin/env bpftrace
/* Monitor top syscalls by process */

tracepoint:raw_syscalls:sys_enter
{
    @syscalls[comm, args.id] = count();
}

interval:s:5
{
    print(@syscalls);
    clear(@syscalls);
}
EOF

sudo bpftrace /tmp/syscall_monitor.bt

```

✓ **Checkpoint:** You can write bpftrace one-liners for quick kernel analysis.

❏ Exercise 13.2: BCC Tools — Pre-built eBPF Observability

Objective: Use BCC's collection of eBPF-based performance tools.

Step 1: Install BCC:

```

sudo apt install -y bpfcc-tools python3-bpfcc

```

Step 2: Use pre-built tools:

```

# Process execution tracing
sudo execsnoop-bpfcc

# File opens
sudo opensnoop-bpfcc

# TCP connection tracing
sudo tcpconnect-bpfcc

# TCP accept tracing
sudo tcpaccept-bpfcc

# Disk I/O latency

```

```

sudo biolatility-bpfcc

# Disk I/O by process
sudo biotop-bpfcc

# Cache hit ratio
sudo cachestat-bpfcc

# Function latency (any kernel function)
sudo funclatency-bpfcc vfs_read

# Stack traces causing block I/O
sudo stackcount-bpfcc blk_account_io_start

# Run queue latency
sudo runqlat-bpfcc

# CPU profiling
sudo profile-bpfcc -F 99 5      # 99 Hz for 5 seconds

```

Step 3: Use BCC for debugging a specific scenario:

```

# Trace why a process is slow:
# 1. What syscalls is it making?
sudo syscount-bpfcc -p $(pgrep my_program)

# 2. Is it waiting on disk?
sudo biolatility-bpfcc -p $(pgrep my_program)

# 3. Is it being preempted?
sudo runqlat-bpfcc -p $(pgrep my_program)

# 4. What functions take the most time?
sudo funclatency-bpfcc -p $(pgrep my_program) '*'

```

Step 4: Answer in `answers.txt` :

1. What makes eBPF safe to run in the kernel?
2. What is the difference between bpftrace and BCC?
3. Name 5 BCC tools and what they measure.

✅ **Checkpoint:** You can use BCC's pre-built eBPF tools for production observability.

LAB 14: Crash Dump Analysis

Mission: "The Kernel Autopsy"

Story: The server kernel panicked last night. The crash dump was saved by `kdump`. Now you must load it into the `crash` utility and perform a full post-mortem analysis — what process was running, what function panicked, and what data was on the stack.

Background

When the kernel panics, **kdump** captures a crash dump by booting a secondary (capture) kernel:

Crash Dump Workflow

1. System runs normally
2. Kernel panic / oops / hang
▼
3. kexec boots the "capture kernel"
(pre-loaded in reserved memory)
▼
4. Capture kernel writes /proc/vmcore
to /var/crash/
▼
5. System reboots normally
6. Analyze with `crash` tool
crash vmlinux /var/crash/vmcore

❏ Exercise 14.1: Setting Up Kdump

Objective: Configure kdump for crash dump collection.

Step 1: Install kdump tools:

```
sudo apt install -y crash kdump-tools kexec-tools linux-image-$(uname -r)-dbg

# Check kdump status
systemctl status kdump-tools
```

Step 2: Configure crash kernel memory reservation:

```
# Edit GRUB
sudo nano /etc/default/grub
# Add to GRUB_CMDLINE_LINUX:
GRUB_CMDLINE_LINUX="crashkernel=256M"

sudo update-grub
sudo reboot

# Verify reservation after reboot
cat /proc/cmdline | grep crashkernel
cat /sys/kernel/kexec_crash_size
```

Step 3: Configure kdump:

```
sudo nano /etc/default/kdump-tools

# Set: USE_KDUMP=1
# Set: KDUMP_COREDIR="/var/crash"

# Restart kdump
```

```
sudo systemctl restart kdump-tools
sudo systemctl status kdump-tools
```

Step 4: Test kdump (WARNING: will crash the system!):

```
# Only do this in a VM!
echo 1 | sudo tee /proc/sys/kernel/sysrq
echo c | sudo tee /proc/sysrq-trigger    # Triggers panic
```

After reboot:

```
ls -la /var/crash/
# Should see a timestamped directory with vmcore
```

✅ **Checkpoint:** Kdump is configured and tested.

🔗 Exercise 14.2: Crash Dump Analysis

Objective: Analyze a kernel crash dump using the `crash` utility.

Step 1: Open the crash dump:

```
# Get the debug kernel (vmlinux with symbols)
sudo apt install -y linux-image-$(uname -r)-dbg

# Open crash dump
sudo crash /usr/lib/debug/boot/vmlinux-$(uname -r) /var/crash/*/vmcore
```

Step 2: Essential crash commands:

```
crash> help                # All commands

# System info
crash> sys                  # System information
crash> mach                 # Machine info (CPU, memory)
crash> log                  # dmesg at time of crash

# Process info
crash> ps                   # Process list at crash time
crash> ps -l                # Long format with state
crash> bt                   # Backtrace of panicking task
crash> bt -a                # Backtrace of ALL CPUs
crash> bt <pid>              # Backtrace of specific process
crash> set <pid>             # Set context to a process

# Stack and registers
crash> bt -f                # Full stack frame
crash> rd -d <addr> 10      # Read memory (decimal)

# Kernel data structures
crash> task                  # Current task_struct
crash> files <pid>           # Open files for a process
crash> net                   # Network info
crash> mount                 # Mounted filesystems
crash> mod                   # Loaded modules
```

```
# Memory analysis
crash> kmem -i          # Memory usage summary
crash> vm <pid>         # Virtual memory layout

# Disassembly
crash> dis <function>   # Disassemble function
crash> dis -l <addr>    # Disassemble with source lines
```

Step 3: Typical crash analysis workflow:

```
crash> log              # 1. Read the panic message
crash> bt               # 2. Get the crash backtrace
crash> bt -f            # 3. Full frame for more detail
crash> dis -l <crash_addr> # 4. See the faulting instruction
crash> set <pid>        # 5. Switch to the crashing process
crash> files            # 6. What files were open?
crash> task             # 7. Task state details
crash> mod              # 8. Any suspicious modules?
```

Step 4: Answer in `answers.txt` :

1. What is kdump and how does it work?
2. What memory must be reserved for the crash kernel?
3. What are the first 3 commands you run in `crash` ?

✅ **Checkpoint:** You can perform a full post-mortem analysis of a kernel crash.

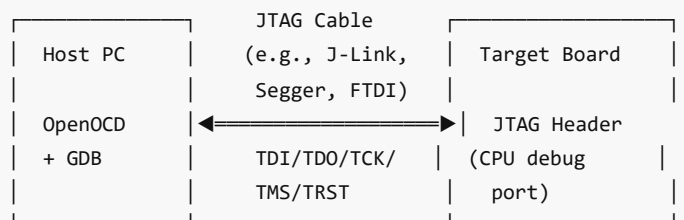
LAB 15: JTAG Debugging

Mission: "The Hardware Debugger"

Story: *The system doesn't even boot. No console, no logs, nothing. The CPU is stuck in early initialization. JTAG is your last resort — it connects directly to the CPU's debug port, letting you halt, step, and examine the processor at the lowest level. This is how firmware engineers debug silicon.*

Background

JTAG (Joint Test Action Group) provides direct hardware access to the CPU:



JTAG can:

- Halt the CPU at any instruction
- Read/write CPU registers
- Read/write memory (including MMIO)
- Set hardware breakpoints
- Flash firmware
- Debug early boot code (before any software is running)

🔧 Exercise 15.1: JTAG Concepts and OpenOCD 🍓

Objective: Understand JTAG debugging principles and the OpenOCD toolchain.

Step 1: Install OpenOCD:

```
sudo apt install -y openocd

# Check available target configs
ls /usr/share/openocd/scripts/target/
ls /usr/share/openocd/scripts/board/
```

Step 2: Understand JTAG signals:

Signal	Direction	Description
TCK	Host→Target	Test Clock
TMS	Host→Target	Test Mode Select
TDI	Host→Target	Test Data In
TDO	Target→Host	Test Data Out
TRST	Host→Target	Test Reset (optional)

Step 3: Example OpenOCD configuration for Raspberry Pi:

```
cat > /tmp/rpi_jtag.cfg << 'EOF'
# JTAG adapter configuration (example: FTDI)
adapter driver ftdi
ftdi vid_pid 0x0403 0x6010
ftdi channel 0
ftdi layout_init 0x0008 0x000b

# Transport
transport select jtag

# Target: BCM2835 (RPi)
set _CHIPNAME bcm2835
jtag newtap $_CHIPNAME cpu -irlen 5 -expected-id 0x07b7617f

target create $_CHIPNAME.cpu arm1176jzs -chain-position $_CHIPNAME.cpu

# Work area in SRAM
$_CHIPNAME.cpu configure -work-area-phys 0x00010000 -work-area-size 0x4000
EOF
```

Step 4: OpenOCD + GDB workflow (conceptual):

```
# Terminal 1: Start OpenOCD
sudo openocd -f interface/ftdi.cfg -f target/bcm2711.cfg

# Terminal 2: Connect GDB
arm-none-eabi-gdb vmlinux
(gdb) target remote localhost:3333
(gdb) monitor halt           # Halt the CPU
(gdb) info registers         # All CPU registers
(gdb) x/10i $pc              # Disassemble at PC
```



```
(gdb) monitor reset halt      # Reset and halt
(gdb) stepi                   # Single-step one instruction
(gdb) monitor resume          # Resume execution
```

Step 5: When to use JTAG vs other debuggers:

Scenario	Tool
Bootloader won't start	JTAG
Kernel doesn't boot	JTAG or KGDB with serial
Kernel panics after boot	KGDB, crash dump
User app crashes	GDB, core dump
Performance issue	perf, ftrace, eBPF

Step 6: Answer in `answers.txt` :

1. What are the 4 required JTAG signals?
2. When is JTAG the ONLY viable debug option?
3. What is OpenOCD's role in JTAG debugging?

✅ **Checkpoint:** You understand JTAG's role in the debugging hierarchy.

PART 3 — NETWORK DEBUGGING

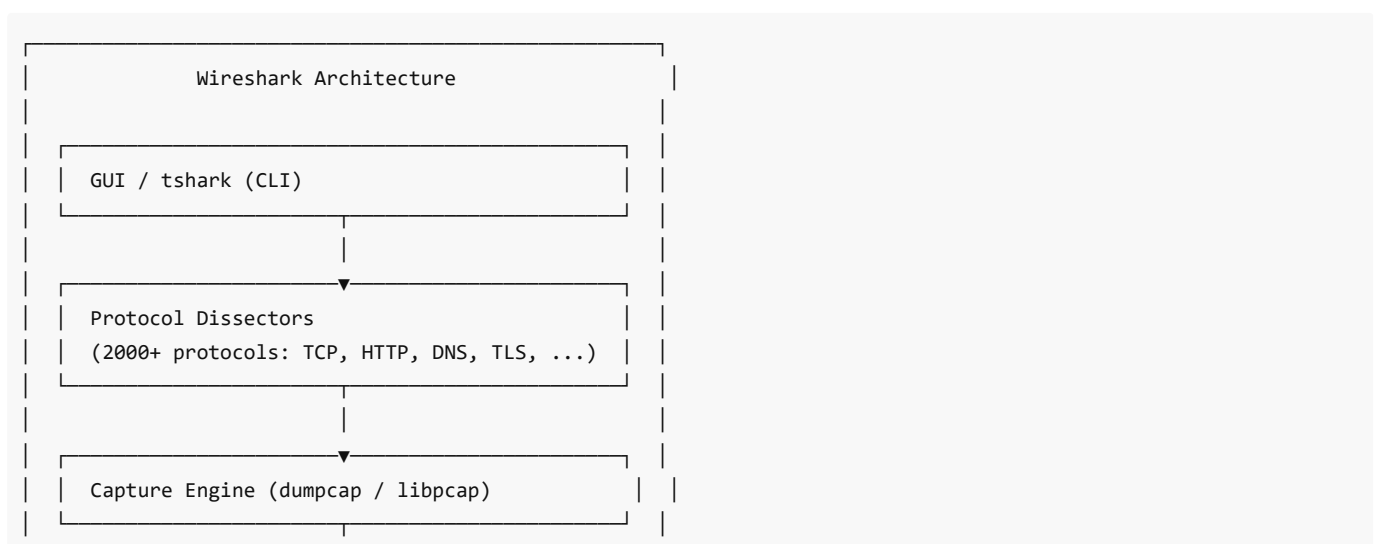
LAB 16: Wireshark — Network Protocol Analyzer

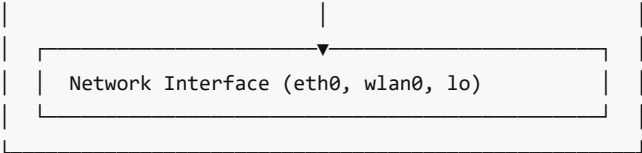
Mission: "The Packet Decoder"

Story: The web application is timing out intermittently. The developers say their code is fine. The network team says the network is fine. Somebody is wrong. Wireshark captures every packet on the wire and lets you see the truth — TCP retransmissions, DNS failures, TLS handshake problems, malformed HTTP headers. The packets don't lie.

Background

Wireshark is the world's most popular network protocol analyzer:





Network Interface (eth0, wlan0, lo)

🔗 Exercise 16.1: Wireshark Capture and Filtering

Objective: Capture network traffic and use display filters. 🍓

Step 1: Install and configure Wireshark:

```
sudo apt install -y wireshark tshark

# Allow non-root capture
sudo dpkg-reconfigure wireshark-common    # Answer "Yes"
sudo usermod -aG wireshark $USER

# Log out and back in for group change to take effect
```

Step 2: Command-line capture with tshark:

```
# List interfaces
tshark -D

# Capture on eth0 (or your interface)
sudo tshark -i eth0 -c 20    # Capture 20 packets

# Capture with filter
sudo tshark -i eth0 -f "port 80" -c 20

# Save to file
sudo tshark -i eth0 -w /tmp/capture.pcap -c 100

# Capture on loopback
sudo tshark -i lo -c 20
```

Step 3: Generate traffic to capture:

```
# In another terminal, generate traffic:
curl -s http://example.com > /dev/null
ping -c 3 8.8.8.8
wget -q http://example.com -O /dev/null

# DNS traffic
dig example.com
nslookup example.com
```

Step 4: Display filters (powerful analysis):

```
# Read a capture file with filters
tshark -r /tmp/capture.pcap

# Filter by protocol
tshark -r /tmp/capture.pcap -Y "http"
tshark -r /tmp/capture.pcap -Y "dns"
```

```
tshark -r /tmp/capture.pcap -Y "tcp"

# Filter by IP
tshark -r /tmp/capture.pcap -Y "ip.addr == 93.184.216.34"
tshark -r /tmp/capture.pcap -Y "ip.src == 192.168.1.100"

# Filter by port
tshark -r /tmp/capture.pcap -Y "tcp.port == 443"
tshark -r /tmp/capture.pcap -Y "tcp.dstport == 80"

# Combine filters
tshark -r /tmp/capture.pcap -Y "http && ip.addr == 93.184.216.34"

# TCP problems
tshark -r /tmp/capture.pcap -Y "tcp.analysis.retransmission"
tshark -r /tmp/capture.pcap -Y "tcp.analysis.duplicate_ack"
tshark -r /tmp/capture.pcap -Y "tcp.analysis.zero_window"

# HTTP specific
tshark -r /tmp/capture.pcap -Y "http.request"
tshark -r /tmp/capture.pcap -Y "http.response.code >= 400"
```

Step 5: Extract useful information:

```
# Show specific fields
tshark -r /tmp/capture.pcap -Y "http.request" \
    -T fields -e ip.src -e http.host -e http.request.uri

# DNS queries
tshark -r /tmp/capture.pcap -Y "dns.flags.response == 0" \
    -T fields -e ip.src -e dns.qry.name

# Protocol hierarchy statistics
tshark -r /tmp/capture.pcap -q -z io,phs

# Conversation statistics
tshark -r /tmp/capture.pcap -q -z conv,tcp

# Endpoints
tshark -r /tmp/capture.pcap -q -z endpoints,ip
```

Step 6: Answer in `answers.txt` :

1. What is the difference between capture filters and display filters?
2. How do you find TCP retransmissions?
3. What display filter shows only HTTP 4xx/5xx errors?

✅ **Checkpoint:** You can capture and filter network traffic with Wireshark/tshark.

🔗 Exercise 16.2: Protocol Analysis

Objective: Analyze TCP handshakes, HTTP transactions, and DNS lookups.

Step 1: Capture a complete HTTP transaction:

```
# Start capture
sudo tshark -i eth0 -w /tmp/http_session.pcap &
TSHARK_PID=$!

# Make HTTP request
curl -v http://example.com 2>/dev/null

# Stop capture
sleep 2
sudo kill $TSHARK_PID 2>/dev/null
```

Step 2: Analyze the TCP 3-way handshake:

```
# Show TCP flags
tshark -r /tmp/http_session.pcap -Y "tcp.flags.syn==1 || tcp.flags.fin==1" \
  -T fields -e frame.number -e ip.src -e ip.dst -e tcp.flags

# The handshake:
# 1. SYN      (client → server)
# 2. SYN,ACK  (server → client)
# 3. ACK      (client → server)
```

Step 3: Follow a TCP stream:

```
# Follow the first TCP stream
tshark -r /tmp/http_session.pcap -q -z follow,tcp,ascii,0
```

Step 4: Analyze DNS resolution:

```
# Capture DNS
sudo tshark -i eth0 -f "port 53" -w /tmp/dns.pcap &
dig example.com
sleep 2
sudo kill $!

# Analyze
tshark -r /tmp/dns.pcap -Y "dns" -V | head -80
```

Step 5: Open in Wireshark GUI for deep inspection:

```
wireshark /tmp/http_session.pcap &
```

In the GUI:

- Right-click a packet → Follow → TCP Stream
- Analyze → Expert Information (find problems)
- Statistics → Protocol Hierarchy
- Statistics → Conversations
- Statistics → IO Graphs

✅ **Checkpoint:** You can analyze complete network transactions at the protocol level.

LAB 17: tcpdump — Command-Line Packet Capture

Mission: "The Server-Side Sniffer"

Story: *The production server doesn't have Wireshark installed — and you can't install it. But tcpdump is already there. It's the field-deployable version of packet capture — lightweight, scriptable, and available on virtually every Linux system.*

Background

tcpdump is the original packet capture tool. It uses the same libpcap library as Wireshark. Capture with tcpdump, analyze with Wireshark — a common workflow.

🔗 Exercise 17.1: tcpdump Essentials

Objective: Master tcpdump for server-side packet capture. 🍓

Step 1: Basic captures:

```
# Capture on default interface
sudo tcpdump -c 10

# Capture on specific interface
sudo tcpdump -i eth0 -c 10

# With human-readable timestamps
sudo tcpdump -i eth0 -tttt -c 10

# Don't resolve hostnames (faster)
sudo tcpdump -i eth0 -n -c 20

# Verbose output
sudo tcpdump -i eth0 -v -c 10
sudo tcpdump -i eth0 -vv -c 10      # More verbose
```

Step 2: Capture filters (BPF syntax — applied at kernel level):

```
# By host
sudo tcpdump -i eth0 host 192.168.1.100

# By network
sudo tcpdump -i eth0 net 192.168.1.0/24

# By port
sudo tcpdump -i eth0 port 80
sudo tcpdump -i eth0 port 443

# By protocol
sudo tcpdump -i eth0 tcp
sudo tcpdump -i eth0 udp
sudo tcpdump -i eth0 icmp

# Combinations
sudo tcpdump -i eth0 'host 192.168.1.100 and port 80'
sudo tcpdump -i eth0 'src 192.168.1.100 and dst port 443'
sudo tcpdump -i eth0 'tcp and port 80 and not host 192.168.1.1'
```

```
# TCP flags
sudo tcpdump -i eth0 'tcp[tcpflags] & tcp-syn != 0'      # SYN packets
sudo tcpdump -i eth0 'tcp[tcpflags] & tcp-rst != 0'      # RST packets
sudo tcpdump -i eth0 'tcp[tcpflags] & (tcp-syn|tcp-fin) != 0'
```

Step 3: Save and analyze:

```
# Save to pcap file (can open in Wireshark)
sudo tcpdump -i eth0 -w /tmp/capture.pcap -c 1000

# Read from file
tcpdump -r /tmp/capture.pcap | head -20

# Read with filter
tcpdump -r /tmp/capture.pcap 'port 80'

# Rotate files (useful for long captures)
sudo tcpdump -i eth0 -w /tmp/capture_%Y%m%d_%H%M%S.pcap \
  -G 3600 -W 24      # New file every hour, keep 24 files
```

Step 4: Show packet contents:

```
# Show packet data in hex and ASCII
sudo tcpdump -i eth0 -X -c 5

# Show only ASCII
sudo tcpdump -i eth0 -A -c 5 port 80

# Show hex only
sudo tcpdump -i eth0 -xx -c 5

# Snap length (capture more/less of each packet)
sudo tcpdump -i eth0 -s 0 -c 5      # Full packets
sudo tcpdump -i eth0 -s 96 -c 5    # First 96 bytes only
```

Step 5: Practical troubleshooting recipes:

```
# DNS troubleshooting
sudo tcpdump -i eth0 -n port 53

# HTTP troubleshooting
sudo tcpdump -i eth0 -A -n 'port 80 and tcp[((tcp[12:1]&0xf0)>>2):4]=0x47455420'
# ↑ This matches "GET " at the start of TCP payload

# SSH connection issues
sudo tcpdump -i eth0 -n port 22

# DHCP troubleshooting
sudo tcpdump -i eth0 -n port 67 or port 68

# ARP traffic
sudo tcpdump -i eth0 -n arp

# Find which process is making connections (combine with ss)
```

```
sudo tcpdump -i eth0 -n 'tcp[tcpflags] == tcp-syn' &
ss -tnp | grep ESTAB
```

Step 6: Answer in `answers.txt` :

1. What is the difference between tcpdump capture filters and Wireshark display filters?
2. How do you capture only SYN packets?
3. How do you save captures for later Wireshark analysis?

✅ **Checkpoint:** You can capture and filter packets on production servers with tcpdump.

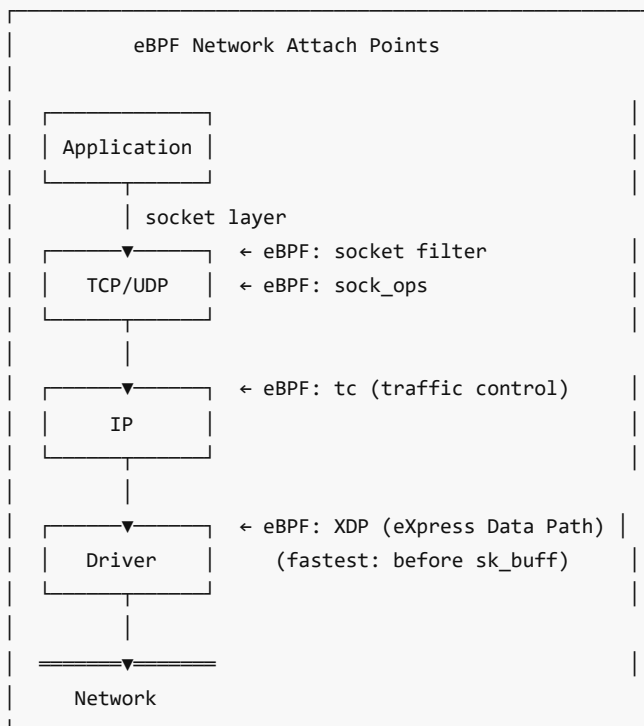
LAB 18: eBPF for Network Debugging

Mission: "The Intelligent Network Observer"

Story: Traditional packet capture (*tcpdump*) captures everything and filters later. With *eBPF*, you can write intelligent filters that run inside the kernel's network stack — counting connections, measuring latency, detecting anomalies — all without copying packets to user space. This is network observability at kernel speed.

Background

eBPF excels at network debugging because it can attach at multiple points in the network stack:



🔧 Exercise 18.1: Network Tracing with BCC Tools

Objective: Use eBPF-based tools for network debugging. 🍓

Step 1: TCP connection monitoring:

```
# Trace new TCP connections
sudo tcpconnect-bpfcc

# In another terminal, generate connections:
```

```
curl -s http://example.com > /dev/null
ssh nonexistent-host 2>/dev/null
```

Output shows: PID, process name, source IP, dest IP, dest port

Step 2: TCP state change tracking:

```
# Track TCP connection state changes
sudo tcpstates-bpfcc

# Watch a connection go through:
# ESTABLISHED → FIN_WAIT1 → FIN_WAIT2 → TIME_WAIT → CLOSE
```

Step 3: TCP retransmission tracking:

```
# Trace TCP retransmissions (sign of network problems)
sudo tcpretrans-bpfcc

# Each retransmission shows:
# Time, PID, source → destination, state
```

Step 4: Network latency measurement:

```
# TCP handshake latency (SYN → SYN-ACK time)
sudo tcpconnlat-bpfcc

# DNS latency
sudo gethostlatency-bpfcc
```

Step 5: Advanced bpftrace network tracing:

```
# Count packets by destination port
sudo bpftrace -e '
kprobe:tcp_v4_connect {
    @connects[comm] = count();
}'

# Trace TCP accept (incoming connections)
sudo bpftrace -e '
kretprobe:inet_csk_accept /retval != 0/ {
    $sk = (struct sock *)retval;
    printf("accept: %s port %d\n", comm,
        $sk->__sk_common.skc_num);
}'

# Network packet size histogram
sudo bpftrace -e '
kprobe:netif_receive_skb {
    @bytes = hist(((struct sk_buff *)arg0)->len);
}'
```

Step 6: Answer in `answers.txt` :

1. What advantage does eBPF have over tcpdump for network debugging?

2. What does `tcpretrans` tell you about the network?
3. What is XDP and where does it run in the network stack?

✅ **Checkpoint:** You can use eBPF for intelligent, low-overhead network debugging.



MINI-PROJECT 1 — GDB Automation Framework

Objective: Build a GDB Python script that automates crash analysis.

Requirements:

1. Write a GDB Python script (`.gdb` or `.py`) that:
 - Automatically loads a core dump
 - Prints the backtrace of all threads
 - Identifies the crashing thread
 - Extracts signal information
 - Dumps local variables in the crash frame
 - Generates a crash report in Markdown format
2. Create a wrapper shell script that runs the analysis
3. Test with at least 3 different crash scenarios (null ptr, overflow, use-after-free)

Deliverables:

- `gdb_crash_analyzer.py` — GDB Python script
 - `analyze_crash.sh` — Wrapper script
 - `test_crashes.c` — Test program with multiple crash modes
 - `sample_report.md` — Example output from the analyzer
-



MINI-PROJECT 2 — Valgrind Test Harness

Objective: Create a test harness that runs programs under Valgrind and generates clean reports.

Requirements:

1. Write a shell script that:
 - Accepts a program and its arguments
 - Runs it under Memcheck, Helgrind, and Callgrind
 - Parses Valgrind XML output
 - Generates a consolidated HTML or Markdown report
 - Returns exit code 1 if any issues found (CI-friendly)
2. Include test programs with intentional bugs for each Valgrind tool
3. Support suppression files for known issues

Deliverables:

- `valgrind_harness.sh` — Main test harness
 - `test_programs/` — Directory of intentional-bug test programs
 - `Suppressions/` — Suppression files
 - `README.md` — Usage instructions
-



MINI-PROJECT 3 — Ftrace Module Profiler

Objective: Build a tool that profiles kernel module function calls using ftrace. 🍓

Requirements:

1. Write a shell script that:
 - Takes a module name as argument
 - Sets up ftrace to trace only that module's functions
 - Measures function call counts and durations
 - Generates a sorted report (most called, slowest)
2. Include a sample kernel module with multiple functions to profile
3. Produce output compatible with FlameGraph tools

Deliverables:

- `module_profiler.sh` — Profiling script
 - `sample_module/` — Kernel module with multiple traceable functions
 - `report_example.txt` — Example profiler output
-



MINI-PROJECT 4 — eBPF Network Dashboard

Objective: Create a real-time network monitoring dashboard using eBPF. 🍓

Requirements:

1. Write bpftrace or BCC scripts that monitor:
 - Active TCP connections per process
 - Connection latency histogram
 - Retransmission count
 - Bytes sent/received per process
 - DNS query latency
2. Create a wrapper that displays all metrics in a formatted terminal output (refreshing)
3. Support filtering by PID or process name

Deliverables:

- `net_dashboard.sh` — Main dashboard script
 - `probes/` — Individual bpftrace/BCC probe scripts
 - `README.md` — Setup and usage instructions
-



MINI-PROJECT 5 — Crash Dump Analysis Toolkit

Objective: Build a toolkit that automates kernel crash dump analysis.

Requirements:

1. Write crash utility batch scripts that:
 - Extract panic message and backtrace
 - List all processes and their states
 - Check for suspicious modules
 - Analyze memory usage at crash time
 - Generate a Markdown report
2. Create a script that triggers controlled panics in a VM for testing
3. Include a reference guide for common crash patterns

Deliverables:

- `crash_analyzer.sh` — Automated crash analysis script
- `crash_batch_commands.txt` — Crash utility batch file

- `trigger_panic.sh` — Controlled panic generator (VM only!)
- `crash_patterns.md` — Reference guide for interpreting crashes

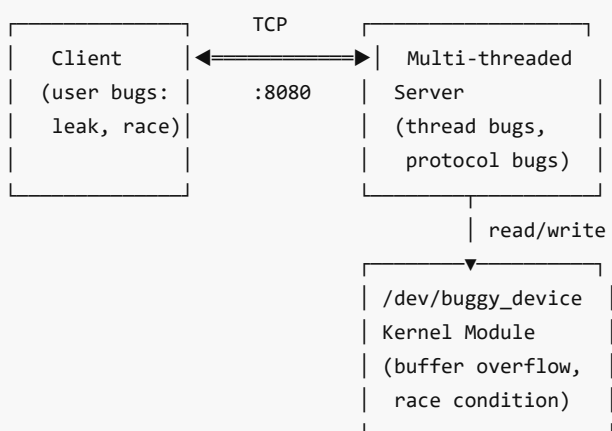
🎓 CAPSTONE PROJECT — Multi-Layer Debugging Investigation

Overview

Build a deliberately buggy multi-component system (client + server + kernel module) and create a comprehensive debugging investigation that uses every tool from this manual.

The system consists of:

- A **kernel module** that provides a `/dev/buggy_device` character device with intentional bugs
- A **multi-threaded server** that reads from the device and serves data over TCP
- A **client** that connects to the server, with its own bugs



Requirements

Phase 1: Build the Buggy System

1. **Kernel module** (`buggy_device.c`):
 - Character device with read/write
 - Intentional buffer overflow in write handler
 - Race condition on shared state (missing lock)
 - Memory leak on specific ioctl command
2. **Server** (`buggy_server.c`):
 - Multi-threaded TCP server
 - Data race on connection counter
 - Memory leak in request handler
 - Use-after-free on client disconnect
3. **Client** (`buggy_client.c`):
 - Connects and exchanges data with server
 - Stack buffer overflow
 - Uninitialized variable use
 - Resource leak (unclosed sockets)

Phase 2: Application-Level Investigation

4. Use **GDB** to:
 - Set breakpoints in client and server

- Catch the segfault in the client
 - Analyze the core dump
 - Debug the multi-threaded server
5. Use **strace** to trace the client-server communication
 6. Use **ASan** to find all memory errors in the client
 7. Use **TSan** to find all data races in the server
 8. Use **Valgrind** (Memcheck) to find all leaks in the server
 9. Use **Valgrind** (Helgrind) to find thread errors

Phase 3: Kernel-Level Investigation

10. Use **ftrace** to trace the kernel module functions
11. Use **perf** to profile the system under load
12. Use **eBPF** (bpftrace/BCC) to:
 - Monitor the module's file operations
 - Track open/close patterns
 - Measure read/write latency
13. Use **dmesg** and **journalctl** to analyze kernel messages
14. If the kernel module causes an oops, analyze with crash or dmesg

Phase 4: Network Investigation

15. Use **tcpdump** to capture client-server traffic
16. Use **Wireshark/tshark** to analyze the protocol
17. Use **eBPF** to track TCP retransmissions and latency
18. Identify any protocol-level bugs from the packet capture

Phase 5: Comprehensive Report

19. Write a detailed investigation report (Markdown) that includes:
 - Bug catalog (each bug: description, tool that found it, fix)
 - Tool comparison (which tool was best for which bug type)
 - Screenshots/logs of tool output
 - Fixed code for all components
 - Lessons learned

Project Structure

```
capstone_debugging/
├── buggy/                # Intentionally buggy code
│   ├── buggy_device.c   # Kernel module
│   ├── buggy_server.c   # Multi-threaded server
│   ├── buggy_client.c   # Client program
│   └── Makefile
├── fixed/               # Corrected code
│   ├── fixed_device.c
│   ├── fixed_server.c
│   ├── fixed_client.c
│   └── Makefile
├── investigation/       # Analysis artifacts
│   ├── gdb_session.txt  # GDB session transcripts
│   ├── strace_output.txt # strace logs
│   ├── asan_report.txt  # ASan output
│   ├── tsan_report.txt  # TSan output
│   ├── valgrind_report.txt # Valgrind output
│   ├── ftrace_log.txt   # ftrace output
│   └── perf_report.txt  # perf analysis
```

```
| |─ ebfprobe/          # bpftrace scripts
| |─ pcap/             # Network captures
| |─ wireshark_analysis.txt
|─ scripts/
| |─ run_all_checks.sh  # Automated bug-finding script
| |─ setup_env.sh       # Environment setup
|─ report.md           # Full investigation report
```

Evaluation Criteria

- All 15+ intentional bugs identified and documented
- Each debugging tool used appropriately for its strength
- Clear explanation of root cause for each bug
- Working fixed code that passes all sanitizers clean
- Investigation report is professional and reproducible